

Documentación técnica oficial — Lunaris Ansenuza

Sistema: Lunaris Ansenuza
Artefacto: `com.lunaris:lunaris-ansenuza:0.0.1-SNAPSHOT`
Versión de referencia: código fuente y migraciones disponibles al 27 de agosto de 2026
Documento: arquitectura, operación, despliegue y mantenimiento

Esta documentación describe exclusivamente lo implementado en este repositorio. Cuando una integración depende de infraestructura externa, se distingue expresamente del código incluido.

1. Resumen ejecutivo y propósito del sistema

Lunaris Ansenuza es una plataforma de gestión de transporte de pasajeros. Centraliza reservas simples y de ida/vuelta, pasajeros y acompañantes, tarifas y promociones, capacidad por servicio, agenda operativa, asignación de choferes, hojas de ruta, comprobantes de pago, facturación y atención conversacional por WhatsApp.

El sistema ofrece tres superficies de interacción:

- 1. **Panel web interno:** administración, agenda, reservas, pasajeros, facturación, choferes, tarifas, configuración, postulaciones y monitoreo del bot.
- 2. **APIs HTTP:** catálogos y operaciones públicas, autenticación de pasajeros, portal, reservas, lista de espera y endpoints administrativos protegidos.
- 3. **Canal WhatsApp:** webhook de entrada, bot de reservas multipaso, avisos, comprobantes, facturas, interacción de choferes y derivación a chat humano en tiempo real.

1.1 Capacidades funcionales principales

Área	Implementación observable
Reservas	Alta manual/API/bot, viaje simple, ida y vuelta con fecha, vuelta abierta, acompañantes, códigos de grupo y vencimiento de pago.
Operación	Agenda por fecha, sentido y horario; capacidad, asignación de choferes, secuencia de recorrido, hoja de ruta y embarque/finalización.

Pasajeros	Identidad, teléfono, CUIL/DNI, domicilio, localidad y saldo a favor.
Comercial	Tarifas por localidad, promociones individuales/masivas, descuento auditado y lista de espera.
Pagos	Carga de comprobantes, confirmación manual y procesamiento opcional de correos IMAP de Mercado Pago con idempotencia y auditoría.
Facturación	Registro único por reserva principal, importe consolidado del grupo, almacenamiento de PDF y envío/reenvío por WhatsApp.
Conversación	Máquina de estados persistente, handlers por paso, pausa del bot, operador asignado, chat WebSocket y recuperación de sesión.
Seguridad	Autenticación web por formulario, Basic/Bearer según superficie, BCrypt y autorización por roles.

1.2 Stack tecnológico real

Componente	Tecnología / versión	Uso
Lenguaje	Java 21	Backend y reglas de negocio.
Framework	Spring Boot 3.5.14	Arranque, configuración y gestión del runtime.
Web	Spring MVC	Controladores HTML y REST.
Persistencia	Spring Data JPA / Hibernate	Entidades, repositorios y transacciones.
Base de datos	PostgreSQL	Persistencia productiva.
Migraciones	Flyway	Evolución y validación del esquema (versionadas hasta V120).
Seguridad	Spring Security	Dos cadenas de filtros, roles, formulario, Basic y filtro Bearer de pasajeros.
UI incluida	Thymeleaf +	Panel web renderizado en

	HTML/JavaScript	servidor.
Tiempo real	Spring WebSocket + SockJS/STOMP	Chat operativo y alertas del bot.
Archivos	Cloudinary Java SDK 1.36.0	Facturas, comprobantes e imágenes; almacenamiento local como fallback en casos definidos.
Mensajería	Meta WhatsApp Cloud API	Texto, botones, documentos, imágenes, plantillas y webhook.
Correo/pagos	Spring Integration Mail + Jakarta Mail	Ingesta IMAPS opcional de notificaciones de Mercado Pago.
API docs	springdoc-openapi 2.8.9	OpenAPI y Swagger UI.
Build	Maven Wrapper	Compilación, pruebas, empaquetado y JaCoCo.
Contenedores	Docker multi-stage, Temurin 21	Build con Maven y ejecución sobre JRE Alpine.
Desarrollo	H2 en memoria, perfil dev	Ejecución local sin PostgreSQL ni Flyway.

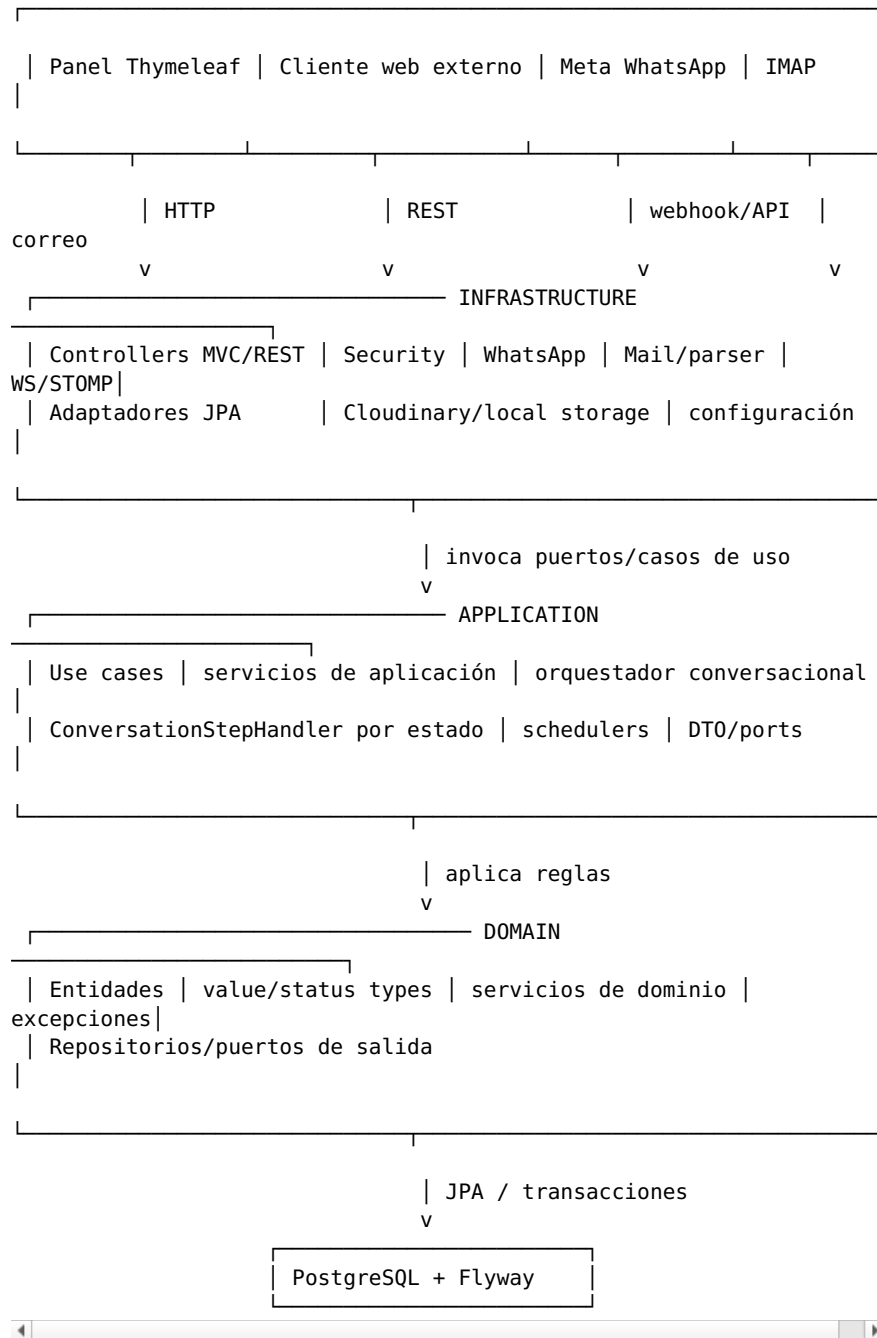
Aclaración sobre React. No hay código React, `package.json` ni pipeline frontend en este repositorio. La interfaz incluida es Thymeleaf. `SecurityConfig` habilita CORS para dominios Vercel y localhost 5173/3000, lo cual permite consumir las APIs desde un frontend externo —posiblemente React—, pero ese cliente no se compila ni despliega desde este proyecto.

2. Arquitectura del sistema

2.1 Estilo arquitectónico

La solución aplica una arquitectura hexagonal pragmática. El núcleo principal se organiza en `domain`, `application` e `infrastructure`; el submódulo `reservation` expresa formalmente puertos de entrada/salida y adaptadores. Algunas entidades JPA residen en `domain/model`, por lo que la separación no es una Clean Architecture estrictamente independiente del framework: el dominio principal conoce anotaciones de Jakarta Persistence y Lombok.

Usuarios / Sistemas externos



La dirección conceptual es infrastructure -> application -> domain. Los puertos desacoplan mensajería, almacenamiento de documentos, chat en vivo y persistencia especializada. Existen dependencias pragmáticas desde algunos casos de uso hacia servicios concretos de infraestructura, especialmente en componentes conversacionales.

2.2 Capa de dominio — domain/

Contiene el modelo y las políticas operativas:

- **Agregados y entidades:** Reservation, Passenger, Invoice, Driver, Vehicle, Promotion, PromotionUsage, ConversationSession, WaitingListEntry, Account, SpecialTrip y eventos de auditoría.
- **Servicios de dominio:**
 - ReservationService: creación de tramos, confirmación, cancelación atómica, créditos, cambios operativos y sincronización de grupos.
 - ReservationCancellationService: decisiones de vuelta recibidas por WhatsApp.
 - PricingAndScheduleService: tarifas, horarios, cálculo de hora estimada y cupos.
 - FleetCapacityService: necesidad y costo de flota adicional.
 - DriverRouteService y TripRouteCalculatorService: orden y sentido del recorrido.
 - SameDayBookingPolicy: restricciones para reservas del mismo día.
 - PromotionService, SystemConfigurationService y WhatsAppConversationWindowService.
- **Excepciones especializadas:** capacidad excedida, promoción vencida/ya usada, reserva ya completada, cierre de reserva del día, entidad no encontrada y validación de dominio.
- **Repositorios de dominio:** interfaces Spring Data para los agregados principales y puertos explícitos para viajes especiales.

Hay dos conceptos de estado distintos en Reservation:

- status (String normalizado por ReservationStatusConverter): estado comercial/pago, con valores canónicos como PENDING_PAYMENT, PAYMENT_RECEIVED, CONFIRMED y CANCELLED.
- travelStatus (TravelStatus): estado operativo del viaje, por ejemplo PENDING, OPEN_RETURN, ROUTE_SENT, IN_PROGRESS, ONBOARD, REALIZED, COMPLETED, CANCELED y NO_SHOW. Los conversores preservan compatibilidad con valores históricos.

2.3 Capa de aplicación — application/

Coordina transacciones y flujos sin ocuparse del transporte HTTP:

Grupo	Ejemplos implementados
Reservas y pagos	CreateReservationUseCase, ConfirmPaymentUseCase, ExpireReservationPaymentUseCase, ProcessPaymentReceiptUseCase.
Facturación	GetBillingPanelUseCase, IssueInvoiceUseCase, InvoicePersistenceService.
Operación	GetDailyOperationSummaryUseCase, GetHojaDeRutaUseCase, OnboardPassengerUseCase, CompleteTripUseCase, ReservationDriverAssignmentService. ScheduleService, servicios de

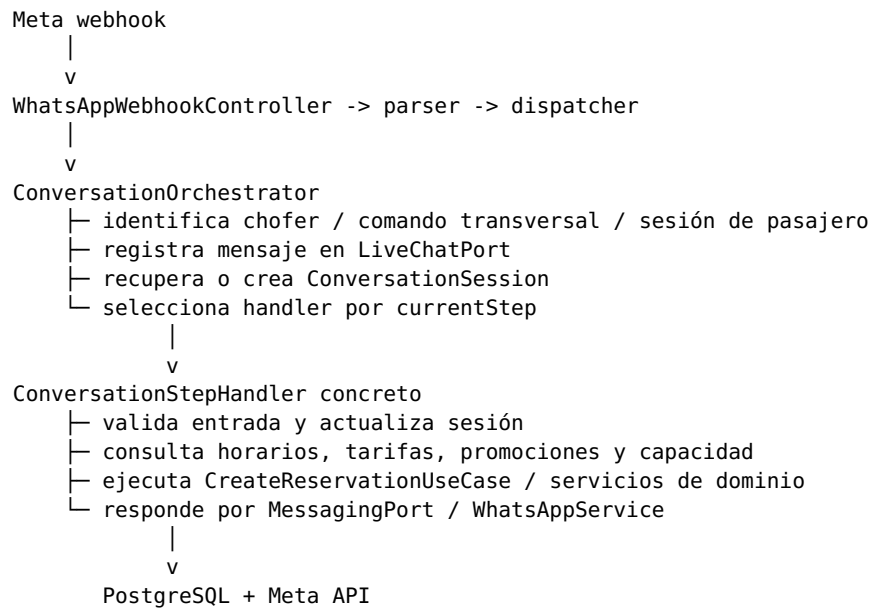
Catálogo	tarifas/localidades y viajes especiales.
Pasajeros	creación, perfil, dirección y OTP/token.
Lista de espera	alta, OTP, conversión y reactivación.
Pagos por correo	ProcessBankEmailUseCase y ProcessBankEmailService.
Conversación	ConversationOrchestrator, ConversationStepHandler y handlers concretos.
Tareas programadas	vencimiento de pagos, limpieza de sesiones y auditoría/recordatorio de vueltas.

Los puertos MessagingPort, InvoiceStoragePort, ReceiptStoragePort, NewsBannerStoragePort, DriverDocumentStoragePort y LiveChatPort abstraen efectos externos. El módulo reservation/application/port agrega contratos explícitos para repositorio y gateway de pagos.

2.4 Capa de infraestructura — infrastructure/

Adaptador	Responsabilidad
Controladores MVC/REST	Entrada HTTP, binding, selección de vista y delegación a servicios/casos de uso.
SecurityConfig	Autenticación, CORS y autorización por endpoint.
Repositorios JPA	Consultas de agenda, locks, persistencia y proyecciones.
WhatsAppService / parser / dispatcher	Integración con Meta Graph API y procesamiento del webhook.
WhatsAppMessagingAdapter	Implementación del puerto de mensajería de aplicación.
CloudinaryInvoiceStorageService	Persistencia y recuperación resiliente de PDFs de factura.
Adaptadores Cloudinary/locales	Comprobantes, banners, facturas y documentación de postulantes.
MercadoPagoImapAdapterConfig	Polling IMAPS, whitelist de remitentes y entrega al caso de uso de pagos.
WebSocketLiveChatAdapter	Persistencia y publicación de chat/alertas a tópicos STOMP.
Adaptadores de auditoría	Ledger de transacciones procesadas y outbox de detecciones de pago.

2.5 Flujo representativo de reserva por WhatsApp



3. Patrones de diseño e integración implementados

3.1 State Machine / Step Handler

El bot persiste el estado en `conversation_sessions.current_step`. Al iniciar, `ConversationOrchestrator` reúne todos los beans `ConversationStepHandler` en un mapa por el valor retornado por `step()`. Cada mensaje se delega al handler correspondiente.

Handlers relevantes incluyen `StartHandler`, `MainMenuHandler`, `AskLocalityHandler`, `AskDestinationHandler`, `AskTripTypeHandler`, `AskDateHandler`, `SelectScheduleHandler`, `AskReturnDateHandler`, `AskAddressTextHandler`, `AskNameHandler`, `AskDniHandler`, `AskCompanionsCountHandler`, `AskPromotionCodeHandler`, `ConfirmationHandler`, `AwaitingPaymentHandler` y los pasos de lista de espera.

Propiedades operativas del patrón:

- La sesión sobrevive entre mensajes y reinicios del proceso porque se almacena en PostgreSQL.
- Un saludo o comando de reinicio vuelve al handler START.
- Un estado desconocido provoca limpieza de datos transitorios y recuperación hacia START.
- Las ubicaciones de WhatsApp se normalizan hacia el paso de domicilio.
- Si `bot_paused=true`, el mensaje queda disponible en el chat humano y no avanza el flujo automático.
- Cada sesión nueva recibe el operador con menor carga mediante

OperationControlService.

- Los comandos de chofer y promoción se resuelven antes del flujo ordinario de pasajeros.

3.2 Fallback y resiliencia de facturas

CloudinaryInvoiceStorageService implementa InvoiceStoragePort y es el adaptador primario:

```
store(PDF)
├─ Cloudinary configurado -> upload raw/public en carpeta
facturas
├─┬─ secure_url válida -> persistir URL HTTPS
│   └─ error/sin URL -> almacenamiento local
└─ Cloudinary no configurado -> almacenamiento local

load(URL)
├─ URL HTTPS -> descarga pública
│   └─┬─ 2xx/3xx -> bytes
│       └─ 401/403 -> generar URL firmada y reintentar
│           └─ otro error -> excepción de recuperación
└─ ruta local -> LocalInvoiceStorageService
```

La descarga HTTP establece un User-Agent de navegador, sigue redirecciones y aplica connect timeout = 5 s y read timeout = 10 s. Los PDFs se suben como recurso raw, tipo upload, acceso público, con sobreescritura controlada. Para una URL protegida, el adaptador extrae public_id y tipo de entrega y solicita a Cloudinary una URL firmada.

El fallback local protege la **escritura** cuando Cloudinary está ausente o falla. Una URL HTTPS que no puede recuperarse no cae a un archivo local porque no existe una correspondencia garantizada; se informa mediante excepción.

3.3 Bloqueo pesimista y concurrencia

El sistema usa @Transactional y LockModeType.PESSIMISTIC_WRITE, que Hibernate traduce al bloqueo de fila soportado por PostgreSQL (SELECT ... FOR UPDATE; PostgreSQL/Hibernate puede elegir variantes compatibles según el contexto). No se codifica literalmente FOR NO KEY UPDATE en las migraciones o queries del repositorio.

Recurso bloqueado	Objetivo
reservation_capacity_locks	Serializar la verificación/consumo de cupo mediante una clave determinista de servicio. La fila se crea con INSERT ... ON CONFLICT DO NOTHING y luego se bloquea.
Reservas individuales y grupos	Confirmación de pago, baja, cambios operativos y sincronización de ida/vuelta sin actualizaciones perdidas. Actualización atómica del saldo a

Pasajero	favor durante cancelaciones.
Promoción	Evitar consumos masivos duplicados.
Chofer	Asignación y secuenciación concurrente de rutas.
Lista de espera	Reclamo/conversión sin vender el mismo cupo dos veces.
Factura	Evitar filas duplicadas; además existe índice único por reservation_id.

La migración V114 crea la tabla de locks de capacidad y delimita la unidad de secuencia por (driver_id, travel_date, departure_schedule, route_direction, route_sequence). La migración V120 consolida y exige una sola factura por reserva.

3.4 Cancelación en cascada de ida/vuelta

ReservationService.cancelReservation realiza una baja lógica atómica:

1. Bloquea la reserva solicitada.
2. Impide cancelar tramos completados o que violen la política temporal.
3. Si la ida ya fue utilizada, cancela únicamente las vueltas disponibles; no revierte el tramo consumido.
4. En los demás casos marca status=CANCELLED y travelStatus=CANCELED.
5. Busca y bloquea los tramos asociados, prioritariamente por bookingGroupCode; para datos históricos reconoce códigos base con sufijos -IDA y -VUELTA.
6. Cancela los gemelos y registra un ReservationEvent por cada baja.
7. Solo acredita saldo si payment_verified=true; suma importes reembolsables y actualiza Passenger.currentBalance bajo lock.

La asociación moderna se persiste en booking_group_code desde V113. El reconocimiento por sufijo mantiene compatibilidad con reservas previas.

3.5 Otros patrones relevantes

- **Ports & Adapters:** almacenamiento, mensajería, chat y pagos se expresan como contratos reemplazables.
- **Strategy:** cada paso conversacional es una estrategia registrada por identificador.
- **Outbox/Ledger:** los correos de pago generan auditoría persistente y una tabla de transacciones procesadas evita reprocesamiento por identificador externo/transacción.
- **Idempotencia:** confirmaciones, reclamos de auditoría de vuelta y factura única toleran ejecuciones concurrentes o repetidas.
- **Schedulers:** vencimiento de pagos, limpieza de sesiones y seguimiento de vueltas se ejecutan fuera del request principal.
- **Fallback local:** facturas y comprobantes cuentan con adaptadores

Tabla	Clave	Contenido y reglas principales
passengers	UUID	Nombre, apellido, CUIL, teléfono, domicilio, localidad y current_balance.
reservations	UUID	Pasajero, chofer, fechas, origen/destino, importes, promoción, pago, estados, horario, secuencia, grupo y sentido. Una fila máxima por reservation_id;

invoices	UUID	número, identidad fiscal, importe consolidado, URL PDF y entrega por WhatsApp.
conversation_sessions	BIGINT identity	Una sesión por teléfono; paso, datos parciales, pausa del bot, operador, horario y reserva.
promotion_usages	UUID	Uso de promoción por teléfono y fecha; FK a promotions.
reservation_events	UUID	Auditoría inmutable de eventos por reserva y actor.
waiting_list_entries	BIGINT identity	Fecha, origen/destino, horario, teléfono, pasajeros, estado, OTP y datos de reactivación/evento.
reservation_capacity_locks	VARCHAR	Filas utilizadas exclusivamente para exclusión mutua de cupos.
processed_payment_transactions	UUID	Ledger idempotente de notificaciones de pago.
payment_audit_outbox	UUID	Detecciones de pago pendientes de auditoría/entrega.

Los IDs UUID de las entidades principales usan `GenerationType.UUID`; varias entidades agregan `UUID.randomUUID()` en `@PrePersist` como defensa antes del save. Las entidades con identidad incremental (`conversation_sessions`, chat y lista de espera) conservan `GenerationType.IDENTITY` conforme al esquema real.

4.3 Reserva y agrupación

- Una reserva pertenece a un pasajero y puede tener un chofer.
- `passenger_count` representa el total de plazas del registro; valores nulos o menores a uno se interpretan como una plaza mediante `getTotalSeats()`.
- Los viajes ida/vuelta normalmente producen dos registros con códigos `<base>-IDA` y `<base>-VUELTA`, unidos por `booking_group_code`.
- `trip_type` distingue `ONE_WAY`, `ROUND_TRIP` y `OPEN_RETURN`.
- La fecha centinela histórica 2099-12-31 identifica una vuelta aún no

programada y se excluye de agenda/viajes confirmados.

- `requires_invoice` se fuerza a `true` en el ciclo de persistencia actual de `Reservation`.
- El total monetario de un tramo es `amount + extraAmount`; descuentos se auditan por separado en `discount_amount` y campos de promoción.

4.4 Agenda operativa: sentido y horarios

El sentido se deriva de origen y destino, normalizando Córdoba sin distinguir acento:

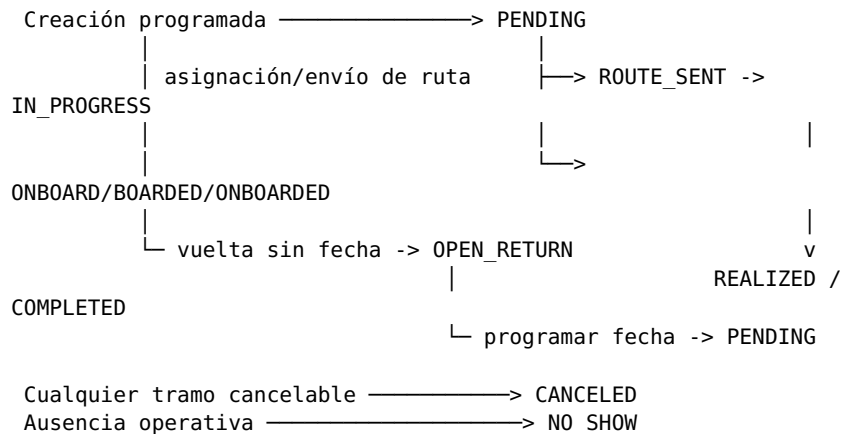
Sentido UI / persistido	Regla	Horarios expuestos en agenda
OUTBOUND / IDA	Origen en pueblos y destino Córdoba o aeropuerto.	03:00, 08:00
RETURN / VUELTA	Origen Córdoba o aeropuerto y destino en pueblos.	12:00, 14:00, 16:00, 17:30

`findActiveManifest` excluye cancelados, vueltas abiertas, completados, realizados y `NO_SHOW`; filtra fecha, prefijo de horario y sentido, y ordena por `route_sequence` y creación. La agenda semanal excluye además fecha centinela y reservas sin pasajeros.

Existe una diferencia deliberada entre superficies: `AgendaViewController` ofrece los seis bloques anteriores, mientras `ScheduleService` publica para el flujo web de regreso 14:00 y 17:30, y `PricingAndScheduleService` mantiene bloques base de salida 03:00 AM y 08:00 AM. Por mantenimiento, no debe asumirse que todas las pantallas ofrecen el mismo catálogo.

4.5 Estados y transiciones operativas

No existe una única tabla declarativa de transiciones; las reglas están distribuidas entre servicios/casos de uso. El flujo observable más relevante es:



CONFIRMED aparece también en `TravelStatus` por compatibilidad, pero la confirmación comercial canónica se registra en `status=CONFIRMED` junto con `payment_verified=true`. No debe confundirse con `travelStatus`.

Para una vuelta abierta, asignar fecha válida y horario la reactiva a `PENDING`. La decisión “más tarde/posponer” recibida por WhatsApp mantiene o devuelve el tramo a `OPEN_RETURN`; “no viaje” ejecuta la cancelación de dominio.

4.6 Facturación

- Solo se factura cuando todos los tramos del grupo están pagados y con `status=CONFIRMED`.
- En un grupo ida/vuelta se suma `amount + extraAmount` de todos los tramos.
- La factura se vincula a la reserva primaria, preferentemente el tramo - IDA.
- No se emite factura fiscal cuando el importe consolidado es cero o negativo.
- El número se genera como F-<año>-<secuencia> usando el conteo de facturas; el índice único y el reintento ante `DataIntegrityViolationException` protegen la concurrencia por reserva.
- La entrega se realiza mediante URL pública del endpoint `/public/invoices/{id}.pdf`; si WhatsApp falla, la factura queda almacenada para reenvío.

4.7 Seguridad y roles

Rol	Acceso principal implementado
ADMIN	Configuración, usuarios, choferes/vehículos, tarifas, viajes especiales, postulaciones y toda la operación.
OPERADOR	Dashboard, agenda, reservas, pasajeros, chat/monitor y endpoints administrativos operativos.
CHOFER	Hoja de ruta y confirmación de asistencia; la URL pública de hoja de ruta también está permitida por diseño.
FACTURACION	Panel y operaciones bajo <code>/facturacion/**</code> .
PASSENGER	Perfil propio mediante autenticación Bearer específica; es una autoridad técnica, no un valor del enum <code>Role</code> .

La autenticación interna usa cuentas persistidas, contraseñas BCrypt y formulario. La cadena /api/** desactiva CSRF y permite sesión IF_REQUIRED, Basic y Bearer de pasajeros; la cadena web mantiene CSRF salvo excepciones explícitas. Catálogos, webhooks, autenticación/portal público, solicitudes de postulantes y descarga pública de facturas tienen permisos específicos.

5. Despliegue, configuración y mantenimiento

5.1 Prerrequisitos

- JDK 21 para ejecución local.
- PostgreSQL accesible para perfiles distintos de dev.
- Credenciales de Meta WhatsApp Cloud API para mensajería real.
- Cuenta Cloudinary para persistencia durable de archivos en producción.
- Buzón IMAPS si se habilita detección de pagos por correo.
- Docker opcional para construir la imagen productiva.

5.2 Variables de entorno

Base de datos y runtime

Variable	Obligatoria en producción	Predeterminada
SPRING_DATASOURCE_URL	Sí	jdbc:postgresql://localhost:5432/
SPRING_DATASOURCE_USERNAME	Sí	Sin valor por defecto.
SPRING_DATASOURCE_PASSWORD	Sí	Sin valor por defecto.
SPRING_JPA_HIBERNATE_DDL_AUTO	No	validate; conservar en producción
SPRING_JPA_SHOW_SQL	No	true; se recomienda false en producción
PORT	No	8080; Render normalmente

WhatsApp y URLs públicas

Variable	Obligatoria	Predeterminada
WHATSAPP_PHONE_NUMBER_ID	Sí en producción	ID del número configurado en Meta.
WHATSAPP_ACCESS_TOKEN	Sí en producción	Token de Graph API. https://lunariis-backend-

LUNARIS_PUBLIC_BASE_URL	Recomendable	nn6s.onrender.com base de enlaces públicos.
LUNARIS_SUPPORT_PHONE	No	Teléfono mostrado/usado para soporte.
PROMOTIONS_AUTHORIZED_OPERATOR_PHONE	Según uso	Teléfono autorizado para comandos promocionales.

La configuración efectiva de agenda contiene además la propiedad histórica `whatsapp.api.token`; no está declarada en `application.yaml` y posee un valor por defecto embebido en `AgendaViewController`. Debe eliminarse/rotarse y unificarse con `WHATSAPP_ACCESS_TOKEN` antes de considerar el manejo de secretos completamente endurecido.

Cloudinary y almacenamiento

Variable	Obligatoria	Predeterminado / función
CLOUDINARY_CLOUD_NAME	Para almacenamiento Cloudinary	Vacío.
CLOUDINARY_API_KEY	Para almacenamiento Cloudinary	Vacío.
CLOUDINARY_API_SECRET	Para almacenamiento Cloudinary	Vacío.
STORAGE_LOCAL_DIR	No	/tmp/comprobantes/
STORAGE_INVOICES_DIR	No	/tmp/facturas/
STORAGE_DRIVER_APPLICATIONS_DIR	No	/tmp/driver-applications/

Los directorios `/tmp` son efímeros en plataformas como Render; Cloudinary debe configurarse para documentos que deban sobrevivir reinicios/despliegues.

Correo IMAP / pagos

Spring Boot permite mapear las propiedades `app.payment.*` mediante variables en mayúsculas:

Variable	Predeterminado	Función
		Habilita el flujo

APP_PAYMENT_IMAP_ENABLED	false	IMAPS.
APP_PAYMENT_IMAP_HOST	imap.gmail.com	Servidor IMAP.
APP_PAYMENT_IMAP_PORT	993	Puerto IMAPS.
APP_PAYMENT_IMAP_POLL_DELAY	30000 ms	Intervalo; el código exige al menos 1000 ms.
PAYMENT_IMAP_USERNAME	Vacío	Usuario del buzón.
PAYMENT_IMAP_PASSWORD	Vacío	Contraseña o app password.
PAYMENT_IMAP_TEST_SENDERS	Vacío	Remitentes de prueba adicionales, separados por coma.
APP_PAYMENT_AUTO_CONFIRM_ENABLED	false	Permite confirmar automáticamente pagos válidos detectados.

El adaptador acepta dominios de Mercado Pago/Mercado Libre y remitentes de prueba autorizados. Configura SSL, timeouts de conexión/lectura de 10 segundos, marca los mensajes como leídos y no los elimina.

Reglas configurables

Variable	Predeterminado	Función
LUNARIS_TRIP_CAPACITY	12	Capacidad base usada por reservas/agenda.
LUNARIS_EXTERNAL_DRIVER_COST	0	Costo de chofer/flota externa.
LUNARIS_OTP_TTL	PT10M	Vigencia OTP de pasajero.
LUNARIS_TOKEN_TTL	PT12H	Vigencia del token Bearer de pasajero.
ADMIN_INITIAL_USERNAME	admin en configuración dev	Usuario inicial.
ADMIN_INITIAL_PASSWORD	admin123 en configuración dev	Contraseña inicial; debe sobrescribirse fuera de

desarrollo.

La lista de espera usa además `lunaris.waiting-list otp-ttl` con default PT5M; puede configurarse por `relaxed binding` como `LUNARIS_WAITING_LIST_OTP_TTL`.

5.3 Ejecución local

Perfil de desarrollo con H2

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

El perfil `dev` usa `jdbc:h2:mem:lunaris_dev`, emulación PostgreSQL, `ddl-auto=update`, consola H2 y Flyway deshabilitado. También provee valores mock de WhatsApp.

Ejecución con PostgreSQL

```
export
SPRING_DATASOURCE_URL='jdbc:postgresql://localhost:5432/lunaris_db'
export SPRING_DATASOURCE_USERNAME='postgres'
export SPRING_DATASOURCE_PASSWORD='cambiar-esta-clave'
export WHATSAPP_PHONE_NUMBER_ID='...'
export WHATSAPP_ACCESS_TOKEN='...'
./mvnw spring-boot:run
```

Flyway se ejecuta al arrancar, con `validate-on-migrate=true` y `out-of-order=true`. Hibernate valida el esquema por defecto.

5.4 Compilación y pruebas

```
# Suite limpia completa
./mvnw clean test

# Empaquetado ejecutando pruebas
./mvnw clean package

# Verificación con reporte JaCoCo
./mvnw verify
```

El JAR queda bajo `target/`. El endpoint de salud expuesto es `/actuator/health`; Swagger UI está en `/swagger-ui.html` y el contrato OpenAPI en `/api-docs`.

5.5 Imagen Docker

```
docker build -t lunaris-ansenuza .
docker run --rm -p 8080:8080 \
-e
SPRING_DATASOURCE_URL='jdbc:postgresql://host.docker.internal:5432/lunaris_db' \
-e SPRING_DATASOURCE_USERNAME='postgres' \
-e SPRING_DATASOURCE_PASSWORD='cambiar-esta-clave' \
-e WHATSAPP_PHONE_NUMBER_ID='...' \
-e WHATSAPP_ACCESS_TOKEN='...' \
lunaris-ansenuza
```

El Dockerfile usa Maven 3.9.6 + Temurin 21 para construir y Temurin 21 JRE Alpine para ejecutar. Actualmente el build de la imagen usa `mvn clean package -DskipTests`; por ello las pruebas deben ejecutarse como etapa separada y obligatoria del CI antes de construir/publicar la imagen.

5.6 Despliegue continuo en Render

El repositorio no contiene `render.yaml`; el servicio debe configurarse desde Render o desde infraestructura externa. La ruta coherente con la implementación es:

```
Push a rama de despliegue
  |
  v
CI: ./mvnw clean test
  | éxito
  v
Render: construir Dockerfile
  |
  ├── conectar PostgreSQL mediante SPRING_DATASOURCE_*
  ├── inyectar secretos WhatsApp/Cloudinary/IMAP
  └── health check: /actuator/health
  v
Arranque -> Flyway migrate/validate -> Spring Boot en $PORT
```

Configuración recomendada del servicio Render:

1. Runtime Docker apuntando al Dockerfile raíz.
2. Auto-deploy solo después del job de pruebas del proveedor Git/CI.
3. Base PostgreSQL administrada y URL JDBC interna.
4. Variables secretas configuradas en el Environment del servicio, nunca en Git.
5. Health check `/actuator/health`.
6. Disco persistente solo si se decide depender de adaptadores locales; la configuración normal debe usar Cloudinary.
7. Webhook Meta dirigido a la URL pública del controlador bajo `/whatsapp/**` según la configuración de la aplicación.

5.7 Operación y mantenimiento

Checklist de despliegue

- ☐ Ejecutar `./mvnw clean test` y conservar el resultado del CI.
- ☐ Validar que todas las migraciones Flyway aplican sobre una copia reciente de producción.
- ☐ Mantener `SPRING_JPA_HIBERNATE_DDL_AUTO=validate`.
- ☐ Verificar `/actuator/health` después del despliegue.
- ☐ Probar webhook y envío de un mensaje WhatsApp no sensible.
- ☐ Verificar carga y descarga de una factura PDF.
- ☐ Confirmar acceso de cada rol con una cuenta de prueba.
- ☐ Revisar logs de schedulers, errores de Cloudinary e ingesta IMAP.

Copias de seguridad

- Respalidar PostgreSQL antes de migraciones y conservar recuperación punto-en-tiempo cuando el proveedor lo permita.
- Cloudinary contiene binarios; PostgreSQL guarda las URLs y metadatos. La restauración debe preservar ambos lados.
- Los archivos bajo /tmp no constituyen backup ni almacenamiento durable.

Observabilidad

- El Actuador expone únicamente health por HTTP.
- La aplicación registra fallos de mensajería, almacenamiento, pagos y decisiones del bot con SLF4J.
- Las tablas reservation_events, processed_payment_transactions y payment_audit_outbox brindan trazabilidad funcional.
- No hay exportador de métricas, tracing distribuido ni agregación de logs configurados en este repositorio.

Riesgos técnicos conocidos observables

1. AgendaViewController contiene un token WhatsApp histórico como valor por defecto; debe rotarse y eliminarse del código.
 2. Los horarios de regreso no son idénticos entre agenda y API/bot; cualquier cambio debe centralizar el catálogo para evitar divergencias.
 3. Reservation.status es texto y convive con un enum operativo más amplio; nuevas transiciones deben probar ambos campos.
 4. El dominio principal está acoplado a JPA; una separación hexagonal estricta requeriría modelos de persistencia y mappers adicionales.
 5. La secuencia de número de factura usa count()+1; la unicidad actual protege por reserva, no necesariamente contra números repetidos entre reservas concurrentes.
 6. El Dockerfile omite tests durante el build; la garantía depende del pipeline previo.
-

6. Mapa de código para mantenimiento

```
src/main/java/com/lunaris/ansenuza/
├── domain/
│   ├── model/                Entidades, enums y converters
│   ├── model/service/        Reglas de dominio
│   ├── repository/           Repositorios principales
│   ├── port/                 Puertos de viajes/tarifas
│   └── exception/            Excepciones especializadas
├── application/
│   └── usecase/               Casos de uso y servicios de
aplicación
│   ├── conversation/steps/    Máquina conversacional por handlers
│   └── payment/              Flujo de correo y confirmación
bancaaria
│   ├── scheduler/            Tareas periódicas
│   └── port/                 Puertos hacia adaptadores
├── infrastructure/
│   └── web/controller/       Entradas MVC/REST y
```

```

ControllerAdvice
├── whatsapp/           Meta Cloud API y webhook
├── storage/            Cloudinary y almacenamiento local
├── adapter/mail|parser/ IMAP y parser Mercado Pago
└── persistence/       Adaptadores/entidades

especializados
├── chat/               WebSocket live chat
├── config/             Seguridad, CORS, async, WS y beans
└── reservation/       Módulo hexagonal explícito de

reservas

src/main/resources/
├── db/migration/       Migraciones Flyway
├── templates/          Vistas Thymeleaf
├── static/             Recursos estáticos
├── application.yaml    Configuración general
├── application.properties IMAP y pool Hikari
└── application-dev.yml Perfil H2/desarrollo

```

7. Criterios para cambios futuros

Todo cambio funcional debe conservar estas invariantes:

- La capacidad se verifica dentro de una transacción y bajo el lock correspondiente.
- Una operación sobre ida/vuelta resuelve el grupo por bookingGroupCode y mantiene compatibilidad con sufijos históricos.
- Una cancelación nunca acredita saldo sin payment_verified=true.
- La agenda excluye estados no operables y vueltas abiertas/no programadas.
- Una factura consolida todos los tramos pagados del viaje y queda vinculada una sola vez a la reserva primaria.
- Los controladores delegan reglas a casos de uso o servicios de dominio.
- Los nuevos endpoints declaran explícitamente su política en SecurityConfig.
- Toda migración es incremental; no se reescriben migraciones ya desplegadas.
- Antes de integrar o desplegar se ejecuta, como mínimo, ./mvnw clean test.